

Zero to LLM

While we wait...

Accounts:

Claude Code Account

[Runpod.io](https://runpod.io) Account

Data & Code:

<https://shorturl.at/g6mjz>

Download Data



Source Code
(Git)



Source Code
(GDrive)



Claude Code Installation Guide:

<https://code.claude.com/docs/en/quickstart>

Before we start...

Our next workshop, “**What’s this ‘AutoResearch’ anyway?**” is next week!

Booth Groups registration will open up on Monday or sooner.

About me



Professional Background

Product manager turned entrepreneur. Currently exploring ideas in the intersection of asset pricing and AI.

Disclaimer

I am NOT an expert in this, still learning, I know a lot less than there is to learn so go easy on me!

"When I can't answer a question, we'll ask our best friend Claude!"

Crash course into LLMs

GPT-2 is **the world's most overachieving autocomplete**. You give it some text, and it predicts the next word.

Then it does it again. And again. Strung together, those predictions look like writing or code.

1. Input sentence

The cat sat on the ____



mat?

floor?

rug?

chair?

sidewalk?

World's most overachieving autocomplete

Tokens. The model first chops your text into small chunks called *tokens*. A token can be a whole word like "cat", a piece of a word like "ing", or even just a single character.

GPT-2 has a fixed vocabulary of about 50,000 of these chunks.

1. Input sentence

The cat sat on the ____



2. Split into tokens, then turn each into a vector of numbers



World's most overachieving autocomplete

Embeddings. Each token gets converted into a long list of numbers, called a *vector*.

Think of this vector as the token's fingerprint. The numbers don't mean anything by themselves, but words used in similar ways end up with similar vectors.

"King" and "queen" end up close together in this number-space.

"King" and "banana" end up far apart.

It also learns these type of relationships:

- King x Female = Queen
- Queen x Male = King

This happens during training.

1. Input sentence

The cat sat on the ____



2. Split into tokens, then turn each into a vector of numbers



World's most overachieving autocomplete

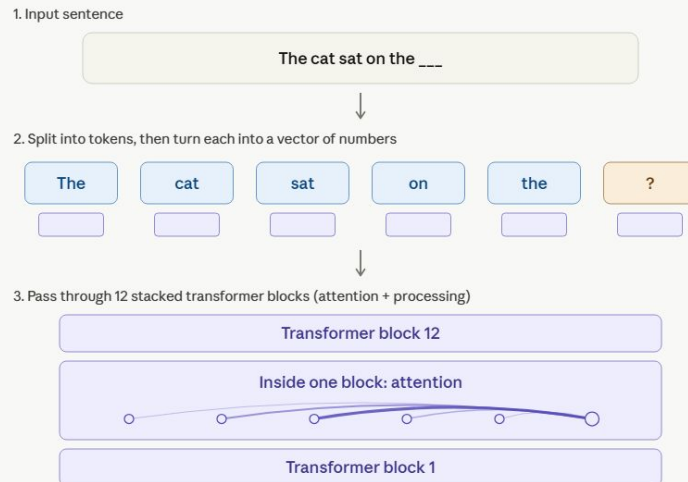
Transformer blocks. This is the engine. GPT-2 small stacks 12 identical layers, each doing two jobs.

The first is *attention*. Every token looks at every previous token and decides which ones matter. For "the cat sat on the ____", the final position pays close attention to "cat" and "sat" because those words tell it what kind of thing should come next.

Thicker lines in the diagram show stronger attention. The model figures this out on its own.

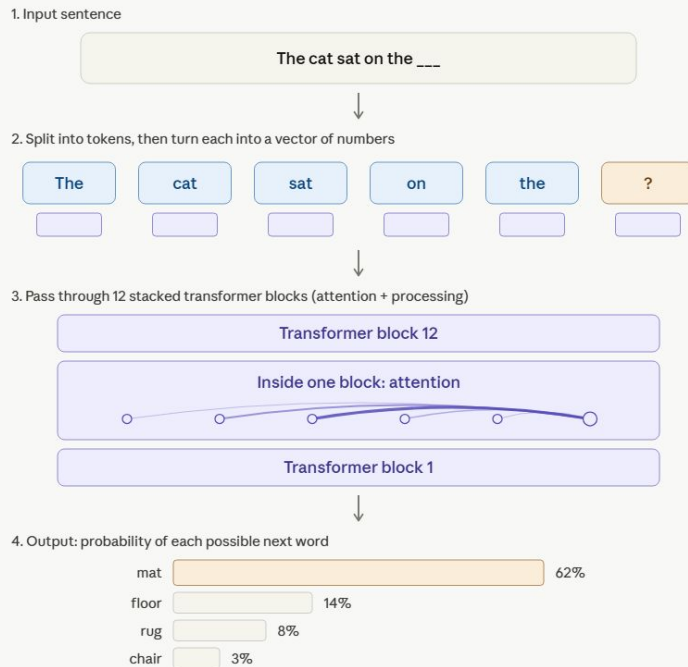
The second is a small neural network (feed forward) that processes the information and decides which numbers to pass on. (Expansion from e.g. 768 -> 3072 and going back to 768)

Each layer adds a level of understanding. Early ones catch grammar. Later ones catch abstract patterns like "this is about an animal, so the answer should be something it could sit on".



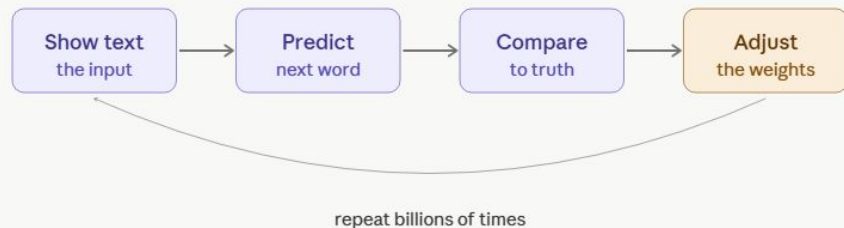
World's most overachieving autocomplete

Output. After the last block, the model produces a probability for every single token in its vocabulary. "mat" might get 62%, "floor" 14%, and so on across all 50,000 candidates. Pick one (usually the highest, sometimes a weighted random choice for variety), add it to the sentence, and run the whole thing again to predict the next token after that.



How does it learn to autocomplete?

Training is conceptually simple. Take a huge pile of text from books, websites, Wikipedia. For every position, hide the next word and ask the model to predict it.



If the model picked the wrong word, that's a mistake. Calculus figures out which of its millions of internal numbers caused the mistake, and each one gets nudged a little in a better direction. GPT-2 did this billions of times on about 40GB of text.

Nobody teaches it facts. Grammar, world knowledge, even style all come from "guess, get corrected, repeat." When training ends, the numbers freeze. Run the model in a loop and you get essays or code.

If you want to learn more

AI & Financial Information: goes deep on the technical side of LLMs, and you learn a lot by building and experimenting. → More fun

ML in Finance: the course to take if you want to understand cutting-edge research at the intersection of ML/AI and finance. Highly recommend if you're serious about applying it. → Less fun

The highest-ROI 3.5 hours of
video you can watch →

Karpathy's
Famous Crash Course



Why (Not) Train Your Own LLM?

The "Why"

- Demystifies the magic a bit, good for mental health!
- It can be very useful!
- It's a lot of fun!

The "Why Not"

- To compete with Anthropic or OpenAI (unless you have billions).
- You don't have concerns around bias, especially look-ahead bias.

My “Why”

Research Projects: Finance

Firms outperforming peers in similar competitive environments may be perceived as "better" by investors, even if their absolute performance lags behind firms in isolation.

The Core Question:

Is 10% growth in tech equal to 10% growth in insurance?

Why my own LLMs?

Claude/ChatGPT/Gemini are biased:

- Reliable backtesting is impossible with Claude because its knowledge cutoff (2026) introduces look-ahead and survivorship bias.

SICs are outdated & static:

- 90% of SIC codes never change; Amazon is still "mail order."
- Firms are multi-faceted (Amazon: Cloud, Retail, Healthcare, etc.)

There is nothing out there that is look-ahead / survivorship bias free that I could use.

So this was my plan:

The Challenge

Extract competitive dynamics from 10-K/Q filings to calculate firm similarity via embeddings.

The Critical Barrier:

Embeddings must be **look-ahead bias free**.

"If extracting embeddings for Amazon in 2010, the model cannot have data after 2009."

Since such a model doesn't exist, I have to train my own.

The Execution Plan

1. Data Extraction

Download 10-K/Q filings for all companies (1996-2026).

2. Rolling Model Training

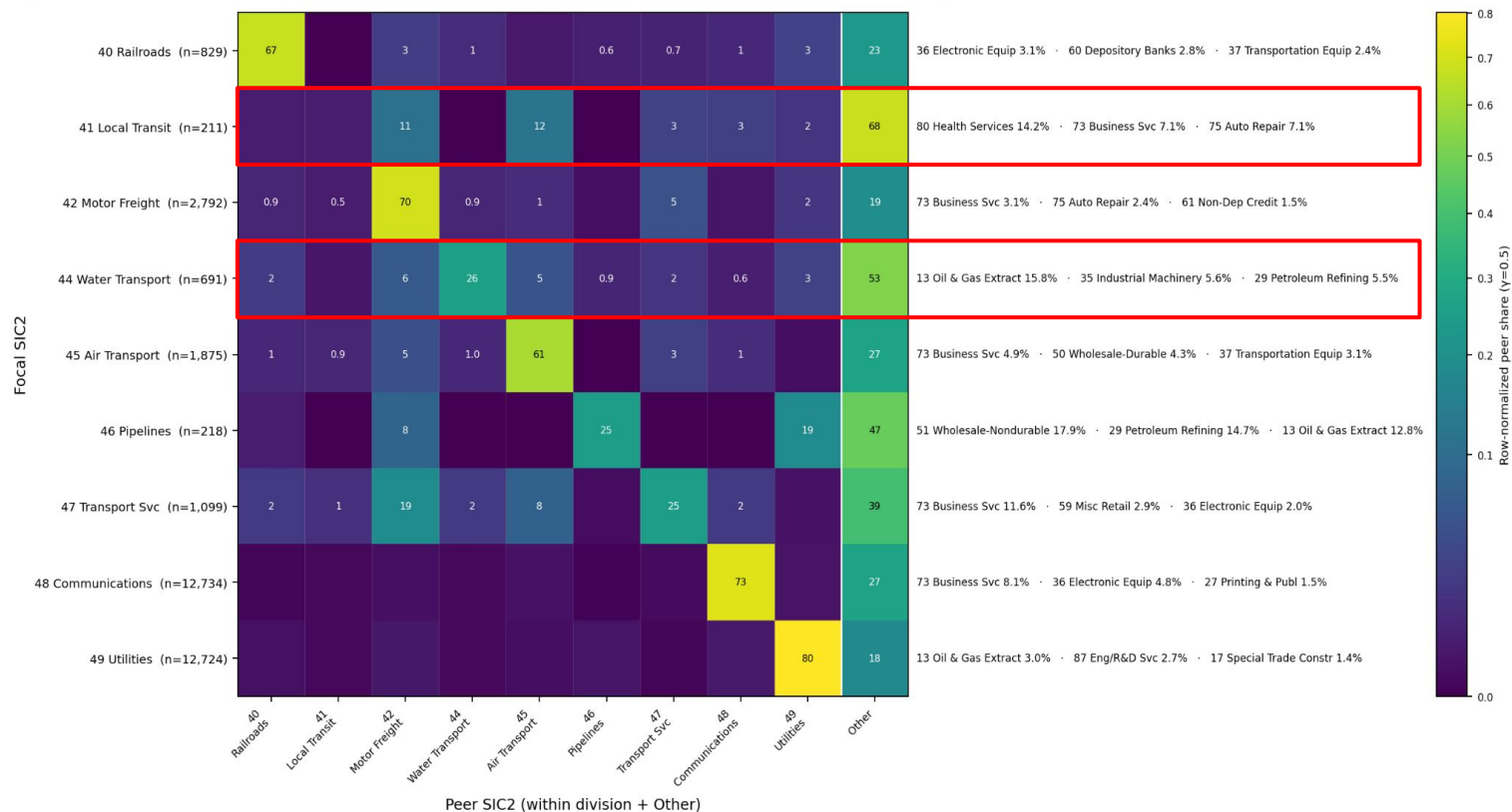
Train a specific model for **each year** using only historical data.

Naming Convention:

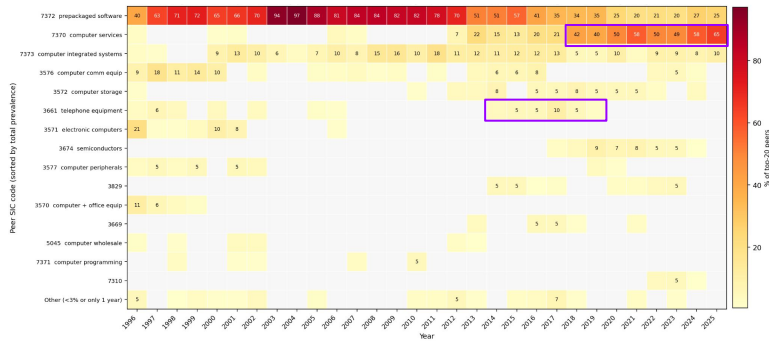
equityAnalystYYYY

***Example:** equityAnalyst2000 was only trained with data between 1996 - 1999.*

How do I know the model found good peers?



Amazon (AMZN)



How was this possible?



Andrej Karpathy

Former cofounder of OpenAI. Released the nanochat/nanogpt library.

github.com/karpathy/nanochat

The Efficiency Leap

GPT-2 Training Cost

2019: >\$40,000

Today: <\$50

Time-to-GPT-2 Leaderboard

Focus on tuning the pretraining stage. Measures wall-clock time to outperform the 1.6B CORE metric.

Metric: DCLM CORE (0.256525)

Compute: 8xH100 GPU node (~\$24/hr)

Let's start...

What you will need:

Claude Code, Runpod Account, Data, and the Code

Claude Code:

<https://code.claude.com/docs/en/quickstart>

macOS, Linux, WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd  
&& del install.cmd
```

If you see The token '&&' is not a valid statement separator, you're in PowerShell, not CMD. If you see 'irm' is not recognized as an internal or external command, you're in CMD, not PowerShell. Your prompt shows PS C:\ when you're in PowerShell and C:\ without the PS when you're in CMD.

Download Data



Source Code
(Git)



Source Code
(GDrive)

